summary_charts

November 4, 2026

1 Summary charts for an OSEM run

OSEM projects and actively trades a portfolio of equities and corporate and government bonds in light of commitments. The basic unit in OSEM is a cash flow. The purpose of this notebook is to show how the different types of cash flows are treated in a run.

1.1 Table of contents

1. Loading the summary csv
   - Run at a glance
2. The initial portfolio
   - Equity
   - Corporate bonds
   - Government bonds
   - Cash
   - Liability profile
3. What is in the summary csv
   - Periodic cash flows
   - Terminal cash flows
   - Cash movement
   - End of period market value
4. Test 1 Return of portfolio
   - Verdict
5. Test 2 No vanishing cash flows
   - Relative residual
   - Verdict

1.2 1. Loading the summary csv

Every run writes its results to `Results.csv` in the output folder. This notebook reads `notebooks/data/Output/Results.csv`, from a run on the small, fixed input set in `notebooks/data/Input`, rather than the live `Output` folder, so its charts and checks do not change with every run of the model. After changing a file in `notebooks/data/Input`, `python notebooks/data/make_results.py` reruns the model and replaces it.

[1]:
```
import pandas as pd
import matplotlib.pyplot as plt
import datetime as dt
from pathlib import Path
```

```
base_folder = next(path for path in [Path.cwd(), *Path.cwd().parents] if (path /
 ↪ "ALM.ini").is_file())
# The notebook inputs, and the results of a run on them
data_folder = base_folder / "notebooks" / "data"
```

[2]:
```
out = pd.read_csv(data_folder / "Output/Results.csv",index_col=0,␣
 ↪parse_dates=[0])
```

1.2.1 Run at a glance

The size and the horizon of the run that produced the summary:

---

[3]:
```
run_summary = pd.DataFrame(
    {"Value": [out.index[0].date(),
               out.index[-1].date(),
               len(out) - 1,
               out["End cash"].iloc[0],
               out["End market value"].iloc[0],
               out["End cash"].iloc[-1] + out["End market value"].iloc[-1]]},
    index=["Modelling date",
           "End of modelling window",
           "Projection periods",
           "Initial cash",
           "Initial market value",
           "Final total value"],
)

display(run_summary)
```

	Value
Modelling date	2023-04-29
End of modelling window	2073-04-29
Projection periods	50
Initial cash	100000.0
Initial market value	116570.8
Final total value	2665640.649477

1.3 2. The initial portfolio

This section shows what the initial inputs of the run were. Currently OSEM can handle four types of financial instruments and one type of liability profile.

1.3.1 Equity

Shares in companies:

[4]:
```
equity = pd.read_csv(data_folder / "Input/Equity_Portfolio.csv",index_col=0)
```

---

[5]:
```
display(equity)
```

Asset_ID	Asset_Type	NACE	Issue_Date	Dividend_Yield	Frequency	\
XXEQ00011250	Equity_Share	A1.4.5	3/12/2021	0.03	1	
XXEQ00021234	Equity_Share	B5.2.0	3/12/2021	0.05	1	
XXEQ00032322	Equity_Share	B8.9.3	3/12/2019	0.04	1	
XXEQ00000014	Equity_Share	A1.4.5	3/12/2021	0.03	1	
XXEQ00000022	Equity_Share	B5.2.0	3/12/2021	0.05	1	

Asset_ID	Units	Market_Price	Terminal	Default_Probability	\
XXEQ00011250	100	123.01	1	0.03	
XXEQ00021234	100	12.12	1	0.03	
XXEQ00032322	100	56.54	1	0.03	
XXEQ00000014	100	77.00	1	0.03	
XXEQ00000022	100	78.00	1	0.03	

Asset_ID	Growth_Rate	Spread_Base	Spread_Individual	Spread_Country	\
XXEQ00011250	0.01	0	0	0	
XXEQ00021234	0.02	0	0	0	
XXEQ00032322	0.04	0	0	0	
XXEQ00000014	0.01	0	0	0	
XXEQ00000022	0.02	0	0	0	

Asset_ID	Spread_Sector	Spread_Stress
XXEQ00011250	0	0
XXEQ00021234	0	0
XXEQ00032322	0	0
XXEQ00000014	0	0
XXEQ00000022	0	0

1.3.2 Corporate bonds

Corporate bonds:

[6]:
```
bonds = pd.read_csv(data_folder / "Input/Corporate_Bond_Portfolio.
↪csv",index_col=0)
```

---

[7]:
```
display(bonds)
```

Asset_ID	Asset_Type	NACE	Issue_Date	Maturity_Date
BE6285455497	Corporate_Bond	C11.0.5	17/03/2016	17/03/2028
FR001400CFW8	Corporate_Bond	K64.1.9	1/9/2022	1/9/2029
FR0013324357	Corporate_Bond	C21.2.0	21/03/2018	21/03/2030
FR0000471930	Corporate_Bond	J61.1.0	28/01/2003	28/01/2033
FR001400DNF5	Corporate_Bond	F42.1.1	30/06/2022	30/06/2042

Asset_ID	Notional_Amount	Coupon_Rate	Spread_Country	Spread_Sector
BE6285455497	100	0.02000	0	0
FR001400CFW8	100	0.03625	0	0
FR0013324357	100	0.01375	0	0
FR0000471930	100	0.08125	0	0
FR001400DNF5	100	0.05375	0	0

Asset_ID	Z_Spread	Spread_Stress	Frequency	Recovery_Rate
BE6285455497	0.0052	0	1	0.4
FR001400CFW8	0.0055	0	1	0.4
FR0013324357	0.0048	0	1	0.4
FR0000471930	0.0077	0	1	0.4
FR001400DNF5	0.0046	0	1	0.4

Asset_ID	Default_Probability	Units	Market_Price
BE6285455497	0.000770	100	93.4649
FR001400CFW8	0.010961	100	99.4582
FR0013324357	0.002621	100	86.4098
FR0000471930	0.011248	100	132.6311
FR001400DNF5	0.024440	100	108.2142

Asset_ID	Issuer
BE6285455497	ANHEUSER-BUSCH INBEV NV
FR001400CFW8	BNP PARIBAS SA
FR0013324357	SANOFI SA
FR0000471930	ORANGE SA
FR001400DNF5	BOUYGUES SA

1.3.3 Government bonds

Government bonds:

[8]:
```
gov_bonds = pd.read_csv(data_folder / "Input/Government_Bond_Portfolio.
↪csv",index_col=0)
```

[9]:
```
display(gov_bonds)
```

Asset_ID	Asset_Type	Issuer_Country	Issue_Date	Maturity_Date	Notional_Amount	Coupon_Rate	Z_Spread	Frequency	Units	Market_Price
DE0001102580	Government_Bond	Germany	15/02/2021	15/02/2032	100	0.000000	0	1	100	73.5773
FR0010171975	Government_Bond	France	25/04/2004	25/04/2055	100	0.040000	0	1	100	108.9074
IT0005398406	Government_Bond	Italy	1/9/2019	1/9/2050	100	0.012250	0	2	100	81.7821
SI0002104121	Government_Bond	Slovenia	3/3/2020	3/3/2081	100	0.006875	0	1	100	34.5930

1.3.4 Cash

Initial cash position:

[10]:
```
cash = pd.read_csv(data_folder / "Input/Cash_Portfolio.csv",index_col=0)
```

[11]:
```
display(cash)
```

Asset_ID	Bank_Account
XEUR00000014	100000

1.3.5 Liability profile

An aggregated view of liabilities displayed as aggregated outflows at future dates:

[12]:
```
liability = pd.read_csv(data_folder / "Input/Liability_Cashflow.
↪csv",index_col=0)
```

[13]:
```
display(liability)
```

```
                Liability_Size
Liability_Date
1/9/2023                     6
2/10/2023                   92
2/11/2023                   93
3/12/2023                    2
3/1/2024                     7
…                          …
19/3/2049                   14
19/4/2049                   28
20/5/2049                   53
20/6/2049                   92
21/7/2049                   41

[306 rows x 1 columns]
```

1.4 3. What is in the summary csv

During the OSEM run, pieces of information are saved into the "Summary" dataframe. The object captures the performance of the portfolio and the size of the cash flows in each step.

Summary object, all columns:

---

[14]:
```
display(out.head(10))
```

```
              Start cash      End cash  Start market value  \
2023-04-29           NaN  1.000000e+05                 NaN
2024-04-29  1.000000e+05  4.365575e-11       116570.800000
2025-04-29  4.365575e-11 -1.818989e-11       221223.638215
2026-04-29 -1.818989e-11 -1.364242e-11       230916.336503
2027-04-29 -1.364242e-11 -5.093170e-11       240268.465408
2028-04-29 -5.093170e-11  0.000000e+00       250260.429902
2029-04-29  0.000000e+00  2.546585e-11       260399.479914
2030-04-29  2.546585e-11  7.275958e-12       271264.379240
2031-04-29  7.275958e-12  1.455192e-11       282119.117898
2032-04-29  1.455192e-11 -1.091394e-10       294788.757292

            After growth market value  End market value  Portfolio return  \
2023-04-29                        NaN     116570.800000               NaN
2024-04-29             117387.810154     221223.638215          0.041793
2025-04-29             223295.686415     230916.336503          0.044117
2026-04-29             232999.703689     240268.465408          0.043661
2027-04-29             242452.598648     250260.429902          0.043634
2028-04-29             231758.956344     260399.479914          0.043411
```

2029-04-29	262747.381434	271264.379240	0.044669
2030-04-29	224967.861184	282119.117898	0.042323
2031-04-29	284657.535928	294788.757292	0.046855
2032-04-29	266102.953650	307897.114438	0.046526

	Dividend cash flow	Coupon cash flow	Terminal cash flow	\
2023-04-29	NaN	NaN	NaN	
2024-04-29	1291.078061	2763.750000	0.0	
2025-04-29	2479.214652	5208.435436	0.0	
2026-04-29	2612.572451	5386.189268	0.0	
2027-04-29	2745.612046	5554.219208	0.0	
2028-04-29	2888.577391	5733.084713	0.0	
2029-04-29	3308.571938	5975.425867	0.0	
2030-04-29	3482.381318	6169.120239	0.0	
2031-04-29	4452.679070	6227.542294	0.0	
2032-04-29	4702.132283	6449.186205	0.0	

	Notional cash flow	…	UL policies in force	UL deaths	\
2023-04-29	NaN	…	0.0	NaN	
2024-04-29	0.000000	…	NaN	NaN	
2025-04-29	0.000000	…	NaN	NaN	
2026-04-29	0.000000	…	NaN	NaN	
2027-04-29	0.000000	…	NaN	NaN	
2028-04-29	20743.861466	…	NaN	NaN	
2029-04-29	0.000000	…	NaN	NaN	
2030-04-29	48125.755157	…	NaN	NaN	
2031-04-29	0.000000	…	NaN	NaN	
2032-04-29	31249.842300	…	NaN	NaN	

	UL lapses	Equity market value	Bond market value	\
2023-04-29	NaN	34667.000000	81903.800000	
2024-04-29	NaN	66477.011962	154746.626253	
2025-04-29	NaN	69955.332628	160961.003875	
2026-04-29	NaN	73415.329992	166853.135416	
2027-04-29	NaN	77130.613406	173129.816497	
2028-04-29	NaN	88222.042535	172177.437379	
2029-04-29	NaN	92727.507736	178536.871504	
2030-04-29	NaN	118399.493971	163719.623927	
2031-04-29	NaN	124859.059664	169929.697628	
2032-04-29	NaN	147140.825669	160756.288769	

	Cash before trade	Remaining liability payments	\
2023-04-29	NaN	14820.0	
2024-04-29	103835.828061	14601.0	
2025-04-29	7620.650088	14534.0	
2026-04-29	7268.761718	13804.0	
2027-04-29	7807.831255	13312.0	
2028-04-29	28640.523570	12587.0	

```
2029-04-29        8516.997806                          11820.0
2030-04-29       57151.256714                          11194.0
2031-04-29       10131.221364                          10645.0
2032-04-29       41794.160788                          10038.0

            UL return credited  Government bond market value  \
2023-04-29                 NaN                    29885.980000
2024-04-29                 NaN                    56815.742319
2025-04-29                 NaN                    59473.756724
2026-04-29                 NaN                    62051.268883
2027-04-29                 NaN                    64812.300623
2028-04-29                 NaN                    73711.635182
2029-04-29                 NaN                    77067.350975
2030-04-29                 NaN                    97958.370590
2031-04-29                 NaN                   102923.596903
2032-04-29                 NaN                    84504.147793

            Corporate bond market value
2023-04-29                  52017.820000
2024-04-29                  97930.883934
2025-04-29                 101487.247151
2026-04-29                 104801.866532
2027-04-29                 108317.515874
2028-04-29                  98465.802197
2029-04-29                 101469.520529
2030-04-29                  65761.253338
2031-04-29                  67006.100724
2032-04-29                  76252.140975

[10 rows x 28 columns]
```

1.4.1 Periodic cash flows

The portfolio generates periodic cash flows from its investment portfolio and outflows from the liabilities that the company committed to. Examples of recurrent inflows are dividends and coupons.

[15]:
```
time_axis = out.index[:-1]

div_axis = out["Dividend cash flow"][:-1]
coup_axis = out["Coupon cash flow"][:-1]
liab_axis = out["Liability cash flow"][:-1]
```

---

[16]:
```
periodic_cf = pd.DataFrame({"Dividend cash flow": div_axis,
                            "Coupon cash flow": coup_axis,
                            "Liability cash flow": liab_axis})
```

```
display(periodic_cf.head(10))
```

	Dividend cash flow	Coupon cash flow	Liability cash flow
2023-04-29	NaN	NaN	NaN
2024-04-29	1291.078061	2763.750000	-219.0
2025-04-29	2479.214652	5208.435436	-67.0
2026-04-29	2612.572451	5386.189268	-730.0
2027-04-29	2745.612046	5554.219208	-492.0
2028-04-29	2888.577391	5733.084713	-725.0
2029-04-29	3308.571938	5975.425867	-767.0
2030-04-29	3482.381318	6169.120239	-626.0
2031-04-29	4452.679070	6227.542294	-549.0
2032-04-29	4702.132283	6449.186205	-607.0

---

[17]:
```
fig, (ax1, ax2, ax3) = plt.subplots(3)

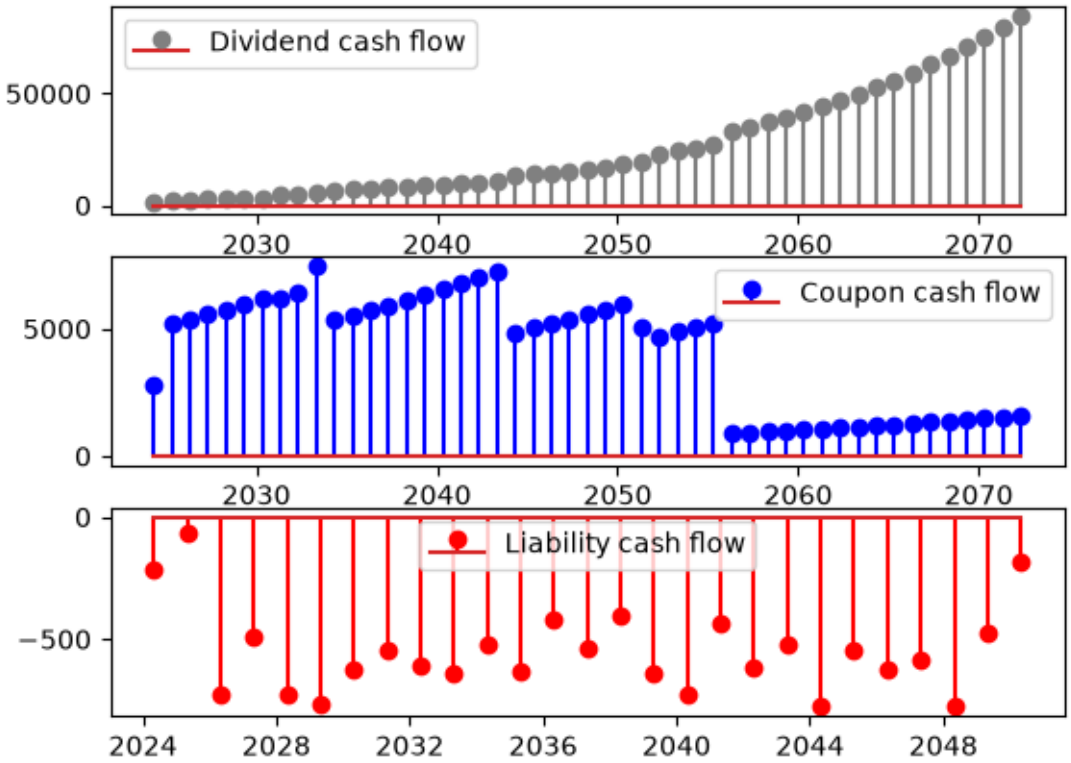
div_mask = div_axis > 0
coup_mask = coup_axis > 0
liab_mask = liab_axis < 0

if div_mask.any():
    ax1.stem(time_axis[div_mask], div_axis[div_mask], linefmt='grey',␣
 ↪label='Dividend cash flow')
    ax1.legend()
else:
    ax1.set_title('No dividend cash flows to plot')

if coup_mask.any():
    ax2.stem(time_axis[coup_mask], coup_axis[coup_mask], linefmt='blue',␣
 ↪label='Coupon cash flow')
    ax2.legend()
else:
    ax2.set_title('No coupon cash flows to plot')

if liab_mask.any():
    ax3.stem(time_axis[liab_mask], liab_axis[liab_mask], linefmt='red',␣
 ↪label='Liability cash flow')
    ax3.legend()
else:
    ax3.set_title('No liability cash flows to plot (e.g. unit-linked mode)')

plt.show()
```



1.4.2 Terminal cash flows

Each asset in OSEM has a terminal value. For fixed income instruments, these are the notional payments at the maturity date. For equities, OSEM assumes that all equity positions are sold at the end of the modelling window at market rates.

[18]:
```
not_axis = out["Notional cash flow"]
term_axis = out["Terminal cash flow"]
```

---

[19]:
```
terminal_cf = pd.DataFrame({"Notional cash flow": not_axis,
                            "Terminal cash flow": term_axis})

display(terminal_cf[(terminal_cf.fillna(0) != 0).any(axis=1)])
```

	Notional cash flow	Terminal cash flow
2028-04-29	20743.861466	0.000000e+00
2030-04-29	48125.755157	0.000000e+00
2032-04-29	31249.842300	0.000000e+00
2033-04-29	36157.946159	0.000000e+00
2043-04-29	58140.119623	0.000000e+00

```
2051-04-29         85920.527522         0.000000e+00
2055-04-29        111822.089074         0.000000e+00
2073-04-29        239775.470048         2.333862e+06
```

---

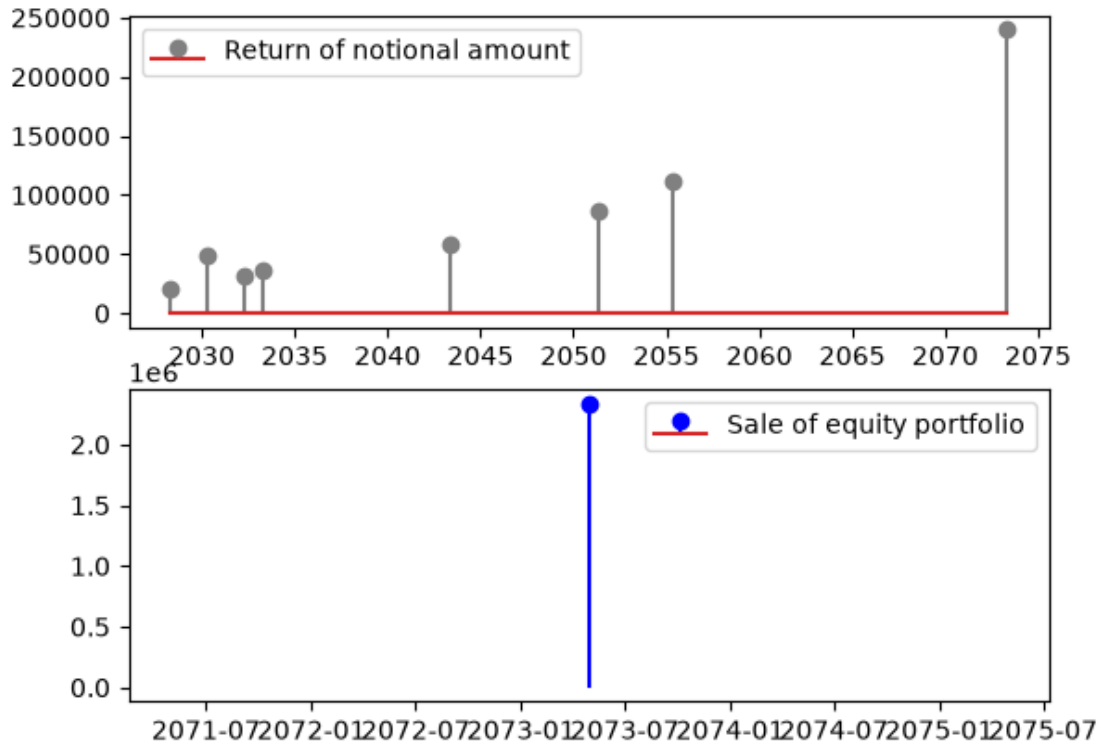
[20]:
```
fig, (ax1, ax2) = plt.subplots(2)

not_mask = not_axis > 0
term_mask = term_axis > 0

if not_mask.any():
    ax1.stem(out.index[not_mask], not_axis[not_mask], linefmt='grey',␣
 ↪label='Return of notional amount')
    ax1.legend()
else:
    ax1.set_title('No notional cash flows to plot')

if term_mask.any():
    ax2.stem(out.index[term_mask], term_axis[term_mask], linefmt='blue',␣
 ↪label='Sale of equity portfolio')
    ax2.legend()
else:
    ax2.set_title('No terminal cash flows to plot')

plt.show()
```



1.4.3 Cash movement

The management of cash between periods is fundamental to OSEM. If the initial cash is high, then in the first period the large cash position is sold in line with the asset allocation rule. This is why the initial cash is represented separately. After the first period, cash swings as the balance of inflows and outflows changes through time.

[21]:
```
x_axis = out.index
end_cash_axis = out["End cash"]
start_cash_axis = out["Start cash"]
```

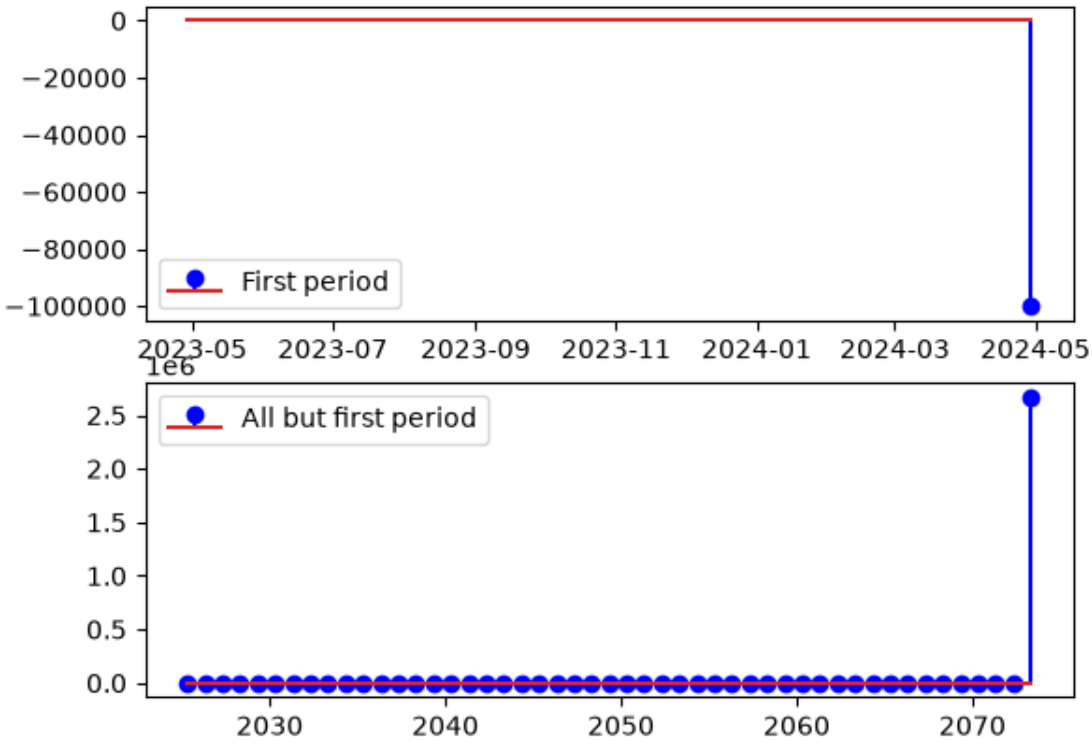
[22]:
```
cash_movement = pd.DataFrame({"Start cash": start_cash_axis,
                              "End cash": end_cash_axis,
                              "Cash movement": end_cash_axis - start_cash_axis})

display(cash_movement.head(10))
```

	Start cash	End cash	Cash movement
2023-04-29	NaN	1.000000e+05	NaN
2024-04-29	1.000000e+05	4.365575e-11	-1.000000e+05
2025-04-29	4.365575e-11	-1.818989e-11	-6.184564e-11
2026-04-29	-1.818989e-11	-1.364242e-11	4.547474e-12
2027-04-29	-1.364242e-11	-5.093170e-11	-3.728928e-11
2028-04-29	-5.093170e-11	0.000000e+00	5.093170e-11
2029-04-29	0.000000e+00	2.546585e-11	2.546585e-11
2030-04-29	2.546585e-11	7.275958e-12	-1.818989e-11
2031-04-29	7.275958e-12	1.455192e-11	7.275958e-12
2032-04-29	1.455192e-11	-1.091394e-10	-1.236913e-10

[23]:
```
fig, (ax1, ax2) = plt.subplots(2)
ax1.stem(x_axis[0:2], end_cash_axis[0:2]-start_cash_axis[0:2], linefmt='b',␣
 ↪label='First period')
ax2.stem(x_axis[2:], end_cash_axis[2:]-start_cash_axis[2:], linefmt='b',␣
 ↪label='All but first period')

ax1.legend()
ax2.legend()
plt.show()
```



1.4.4 End of period market value

The market value is recorded three times per period: at the start, after the portfolio has grown but before any trading, and at the end once the trading is done.

[24]:
```
start_mv_axis = out["After growth market value"]
end_mv_axis = out["End market value"]
```

---

[25]:
```
market_value = pd.DataFrame({"Start market value": out["Start market value"],
                             "After growth market value": start_mv_axis,
                             "End market value": end_mv_axis})

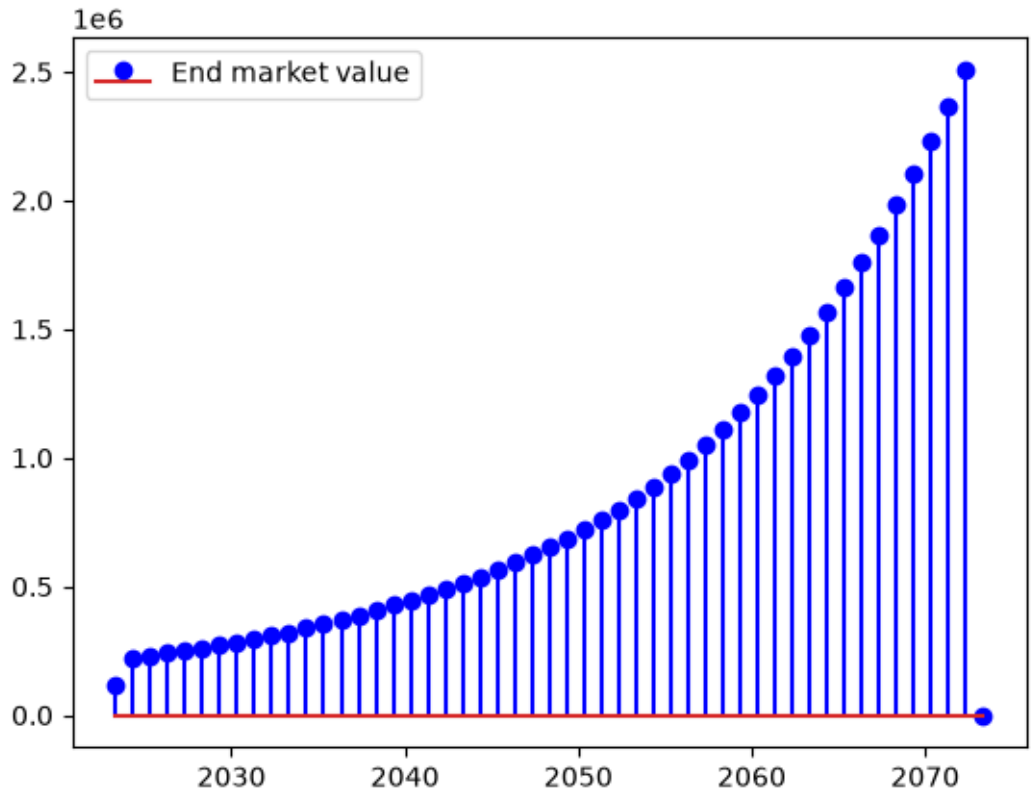
display(market_value.head(10))
```

	Start market value	After growth market value	End market value
2023-04-29	NaN	NaN	116570.800000
2024-04-29	116570.800000	117387.810154	221223.638215
2025-04-29	221223.638215	223295.686415	230916.336503
2026-04-29	230916.336503	232999.703689	240268.465408
2027-04-29	240268.465408	242452.598648	250260.429902

2028-04-29	250260.429902	231758.956344	260399.479914
2029-04-29	260399.479914	262747.381434	271264.379240
2030-04-29	271264.379240	224967.861184	282119.117898
2031-04-29	282119.117898	284657.535928	294788.757292
2032-04-29	294788.757292	266102.953650	307897.114438

---

[26]:
```
fig, ax = plt.subplots(1)
ax.stem(out.index, end_mv_axis, linefmt='b' , label='End market value')
ax.legend()
plt.show()
```



1.5 4. Test 1 Return of portfolio

The purpose of this test is to verify if the reported return of the portfolio is equal to the increase or decrease of the total market value of the portfolio.

Cash paid out by the assets during the period leaves the portfolio before it is repriced: a maturing bond, for example, drops out of the market value entirely. Those flows have to be added back, otherwise the recomputed return would report them as a loss. This is the same asset income that

the model itself uses in `portfolio_total_return`.

Terminal value sales are excluded for the same reason as in Test 2: the equities sold at the end of the modelling window are still inside "After growth market value" when the return is calculated, so adding their proceeds would count them twice.

The test performs the following check

$$r_i = (MV_i^{ag} + div_i + coup_i + not_i)/MV_i^{start} - 1$$

where r_i is the reported "Portfolio return" and the other symbols are as defined in Test 2.

[27]:
```
# Asset income has to be added back to the end market value. The flows are␣
 ↪removed from the
# asset cash-flow matrices before repricing, so without them a maturing bond␣
 ↪looks like a loss
# of its full notional. Terminal sales are excluded: those equities are still␣
 ↪in the market
# value at this point, so their proceeds would be counted twice.
asset_income = (out["Dividend cash flow"].fillna(0)
                + out["Coupon cash flow"].fillna(0)
                + out["Notional cash flow"].fillna(0))

return_calculated = ((out["After growth market value"] + asset_income) /␣
 ↪out["Start market value"] - 1)[1:]

test_1 = out["Portfolio return"][1:] - return_calculated

# Both sides are returns, so the difference is already dimensionless and a␣
 ↪plain absolute
# tolerance is enough. Float64 noise on a ratio of ~1e7 sized values lands␣
 ↪around 1e-16.
test_1_tolerance = 1e-10
```

---

[28]:
```
largest_return_diffs = (test_1.abs()
                        .sort_values(ascending=False)
                        .head(5)
                        .rename("Absolute return difference")
                        .to_frame())

display(largest_return_diffs)
```

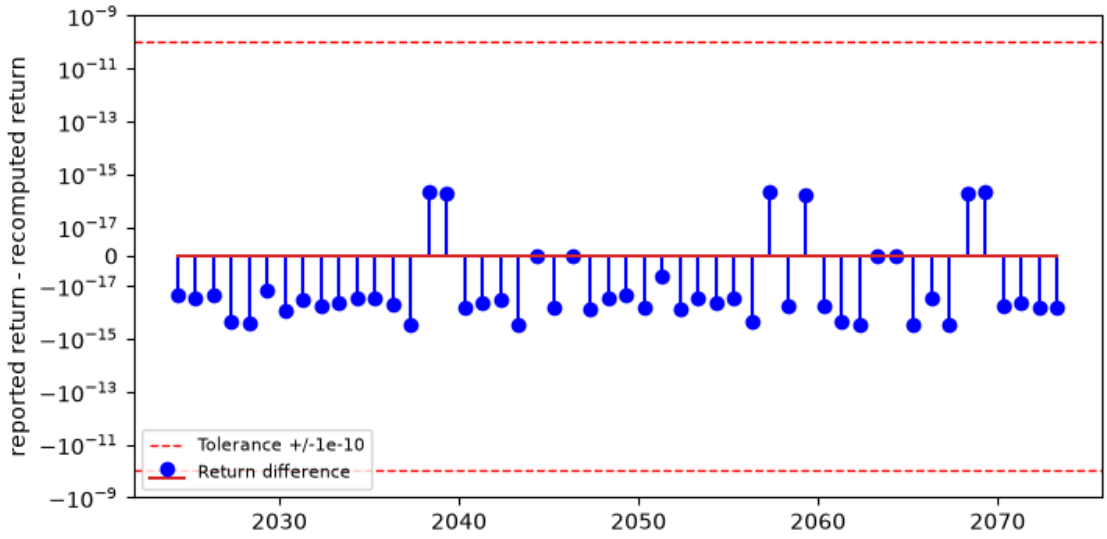
	Absolute return difference
2037-04-29	3.122502e-16
2043-04-29	3.053113e-16
2065-04-29	2.914335e-16

```
2067-04-29                2.914335e-16
2062-04-29                2.844947e-16
```

---

[29]:
```
fig, ax = plt.subplots(1, figsize=(8, 4))
ax.stem(out.index[1:], test_1, linefmt='b', label='Return difference')
ax.axhline(test_1_tolerance, color='r', linestyle='--', linewidth=1,
           label=f'Tolerance +/-{test_1_tolerance:.0e}')
ax.axhline(-test_1_tolerance, color='r', linestyle='--', linewidth=1)

# Symmetric log keeps the differences and the tolerance lines, several orders␣
 ↪of magnitude
# apart, both legible on one axis
ax.set_yscale('symlog', linthresh=1e-17)
ax.set_ylim(-10 * test_1_tolerance, 10 * test_1_tolerance)
ax.set_ylabel('reported return - recomputed return')
ax.legend(loc='lower left', fontsize=8)
plt.show()
```



1.5.1 Verdict

---

[30]:
```
# The first period is dropped by the [1:] slice above: it has no "Start market␣
 ↪value" to
# measure a return against.
return_diffs = test_1.dropna()
```

```
return_breaches = return_diffs[return_diffs.abs() > test_1_tolerance]

print(f"Periods tested         : {len(return_diffs)}")
print(f"Max |return difference|: {return_diffs.abs().max():.3e}  (tolerance␣
 ↪{test_1_tolerance:.0e})")
print()

if return_breaches.empty:
    print("TEST 1 PASSED - reported return matches the recomputed return")
else:
    print(f"TEST 1 FAILED - {len(return_breaches)} period(s) out of tolerance:")
    display(pd.DataFrame({"Reported return": out["Portfolio␣
 ↪return"][return_breaches.index],
                          "Recomputed return":␣
 ↪return_calculated[return_breaches.index],
                          "Difference": return_breaches}))

assert return_breaches.empty, (
    f"Test 1 failed in {len(return_breaches)} period(s): "
    f"{[d.strftime('%Y-%m-%d') for d in return_breaches.index]}"
)
```

```
Periods tested         : 50
Max |return difference|: 3.123e-16  (tolerance 1e-10)

TEST 1 PASSED - reported return matches the recomputed return
```

1.6 5. Test 2 No vanishing cash flows

The purpose of this test is to verify if the inflows and outflows balance at every period. This test shows that all types of cash flows are accounted for correctly and that there is no "leakage".

Starting values are

- Start cash
- After growth market value

Cash flows occurring during the period and accumulating inside the bank account are

- Coupon payments
- Dividend payments
- Notional payments of fixed income assets

Flows between the company and its policyholders also go through the bank account. Which ones depend on the liability mode:

- `cashflow` mode: liability payments
- `unit_linked` mode: gross premiums received, and death benefits and surrender values paid

Fees are not a separate term. The entry fee is part of the gross premium the bank account receives, and the admin fee is taken from the policy accounts; both are recorded in the company account, but neither moves cash on its own.

The end values are

- End cash value
- End market value

Terminal value sales are deliberately **not** part of this balance. The equity positions sold at the end of the modelling window are still inside "After growth market value" when it is recorded, because they are liquidated only after the period return is calculated. The sale therefore converts market value into cash instead of adding new income, and counting it as an inflow would double count it. Notional payments are different: matured bonds reprice to zero before "After growth market value" is taken, so that cash really has left the portfolio.

The test performs the following check

$$C_i^{start} + MV_i^{ag} + CF_i^a - CF_i^l = C_i^{end} + MV_i^{end}$$

where

$$CF_i^a := div_i + coup_i + not_i$$

and

- i is the calculation period
- C_i^{start} is the initial cash position
- MV_i^{ag} is the market value after portfolio growth but before any trading
- CF_i^l are the net outflows to the company's commitments: liability payments in `cashflow` mode, or death benefits plus surrender values less gross premiums in `unit_linked` mode
- C_i^{end} is the final cash position
- MV_i^{end} is the final market value of the portfolio
- div_i is the accumulated dividend inflows
- $coup_i$ is the accumulated coupon inflows
- not_i is the accumulated inflows from matured fixed income instruments (bonds)

Note that the asset growth occurs before trading, therefore "After growth market value" is used as the starting value.

[31]:
```
# The full columns are used here, not the [:-1] slices built for the charts␣
 ↪above. Mixing a
# truncated series with a full one makes pandas align on the index and turn the␣
 ↪last period
# into NaN, which would silently drop the terminal period from the test.
cf_assets = (out["Coupon cash flow"].fillna(0)
             + out["Dividend cash flow"].fillna(0)
             + out["Notional cash flow"].fillna(0))
```

```
# Flows to and from policyholders, signed as they enter the bank account: liability payments in
# cashflow mode, gross premiums in and death and surrender payments out in unit_linked mode. The
# columns of the other mode are empty, so the same sum covers both.
cf_commitments = (out["Liability cash flow"].fillna(0)
                  + out["UL gross premium cash flow"].fillna(0)
                  + out["UL mortality cash flow"].fillna(0)
                  + out["UL lapse cash flow"].fillna(0))

test_2 = start_cash_axis + start_mv_axis + cf_assets + cf_commitments - end_mv_axis - end_cash_axis
```

1.6.1 Relative residual

The absolute residual grows with the portfolio, so a fixed money tolerance would be too tight at the start of the run and too loose at the end. Dividing by the size of the terms that went into the balance puts every period on the same footing and turns the residual into a relative error, directly comparable to the ~1e-16 precision of a 64 bit float.

The terminal period is the noisiest, because it is the only one in which the equity portfolio is valued twice by two different code paths: "After growth market value" compounds the share price period by period, while "Terminal cash flow" raises the starting price to a single power over the whole modelling window. The two agree to a handful of ulps, not bit for bit.

[32]:
```
balance_terms = out[["Start cash", "After growth market value", "Coupon cash flow",
                     "Dividend cash flow", "Notional cash flow", "Liability cash flow",
                     "UL gross premium cash flow", "UL mortality cash flow", "UL lapse cash flow",
                     "End market value", "End cash"]].fillna(0).abs().sum(axis=1)

# where() guards against a zero denominator, which would otherwise give an infinite residual
test_2_rel = test_2 / balance_terms.where(balance_terms != 0)

# Float64 carries ~1e-16 of relative precision and the balance chains together a few dozen
# operations, so anything up to a few hundred ulps is rounding rather than a leak.
test_2_tolerance = 1e-12
```

---

[33]:
```
largest_residuals = (pd.DataFrame({"Relative residual": test_2_rel,
                                   "Absolute residual": test_2})
```

```
                    .dropna()
                    .sort_values("Relative residual", key=lambda s: s.abs(),␣
 ↪ascending=False)
                    .head(5))

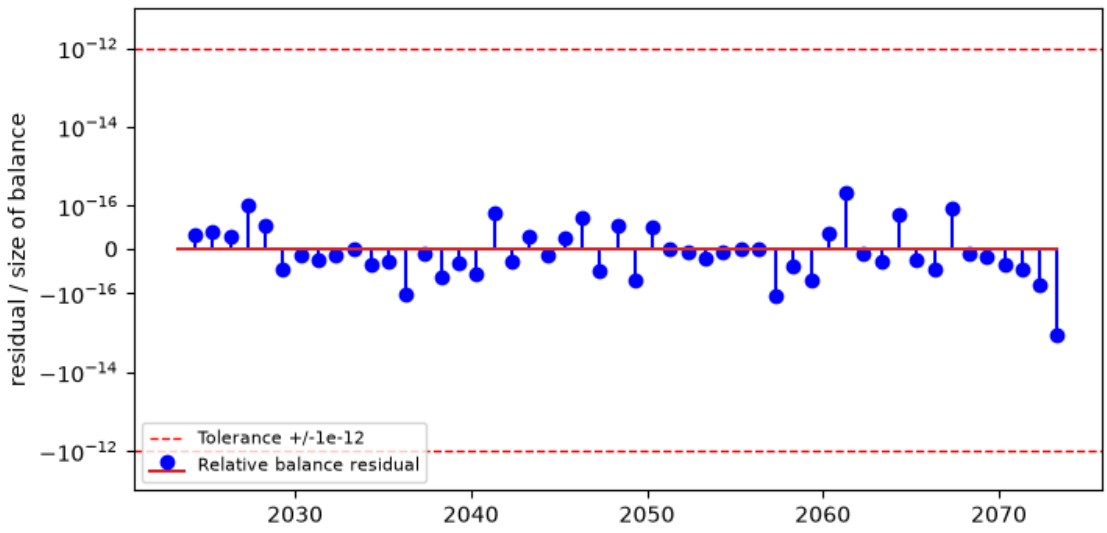
display(largest_residuals)
```

	Relative residual	Absolute residual
2073-04-29	-1.135486e-15	-6.053597e-09
2061-04-29	2.097256e-16	5.529728e-10
2057-04-29	-1.110274e-16	-2.328306e-10
2036-04-29	-1.077389e-16	-8.003553e-11
2027-04-29	1.015577e-16	5.093170e-11

---

[34]:
```
fig, ax = plt.subplots(1, figsize=(8, 4))
ax.stem(x_axis, test_2_rel, linefmt='b', label='Relative balance residual')
ax.axhline(test_2_tolerance, color='r', linestyle='--', linewidth=1,␣
 ↪label=f'Tolerance +/-{test_2_tolerance:.0e}')
ax.axhline(-test_2_tolerance, color='r', linestyle='--', linewidth=1)

# Symmetric log keeps the residuals and the tolerance lines, three orders of␣
 ↪magnitude apart,
# both legible on one axis
ax.set_yscale('symlog', linthresh=1e-16)
ax.set_ylim(-10 * test_2_tolerance, 10 * test_2_tolerance)
ax.set_ylabel('residual / size of balance')
ax.legend(loc='lower left', fontsize=8)
plt.show()
```



1.6.2 Verdict

[35]:
```
# The first row is the opening position rather than a projection period: it has␣
 ↪no "Start cash"
# and no "After growth market value", so there is nothing to balance and it is␣
 ↪dropped.
residuals = test_2_rel.dropna()
breaches = residuals[residuals.abs() > test_2_tolerance]

print(f"Periods tested          : {len(residuals)}")
print(f"Max |relative residual| : {residuals.abs().max():.3e}  (tolerance␣
 ↪{test_2_tolerance:.0e})")
print(f"Max |absolute residual| : {test_2.abs().max():.3e}")
print()

if breaches.empty:
    print("TEST 2 PASSED - no vanishing cash flows")
else:
    print(f"TEST 2 FAILED - {len(breaches)} period(s) out of tolerance:")
    display(pd.DataFrame({"Relative residual": breaches,
                          "Absolute residual": test_2[breaches.index]}))

assert breaches.empty, (
    f"Test 2 failed in {len(breaches)} period(s): "
    f"{[d.strftime('%Y-%m-%d') for d in breaches.index]}"
)
```

```
Periods tested          : 50
Max |relative residual| : 1.135e-15  (tolerance 1e-12)
Max |absolute residual| : 6.054e-09

TEST 2 PASSED - no vanishing cash flows
```